Appendix A

Answers to the Test Your Knowledge Questions

This appendix has the answers to the questions in the *Test your knowledge* section at the end of each chapter.

Chapter 1 – Hello, C#! Welcome, .NET!

Exercise 1.3 – Test your knowledge

Suggested answers to these questions:

1. Which of the three main code editors or IDEs discussed in this chapter is Windows-only, and which are cross-platform?

   Answer: Visual Studio is Windows-only. Visual Studio Code and JetBrains Rider are cross-platform and run on Windows, macOS, and Linux.

2. What is the difference between the .NET SDK and the .NET Runtime?

   Answer: The .NET SDK contains the tools needed to create, build, test, and publish .NET applications. It includes the compiler, command-line tools, project templates, and a .NET Runtime. The .NET Runtime contains only the components needed to run an already-built .NET application.

3. What is a `.csproj` file used for?

   Answer: A `.csproj` file is the project file for a C# project. It defines settings such as the target framework, project type, nullable-reference-type behavior, package references, project references, files and resources, and build options.

4. What is a solution, and why might it contain more than one project?

   Answer: A solution is a container used to organize one or more related projects. A solution might contain several projects because a complete application can be divided into components such as a console app, a class library, a web app, and a unit-test project.

5. Why might you choose **Start Without Debugging** instead of **Start** when running a console app?

   Answer: Start Without Debugging runs the application without attaching the debugger. This can start faster, avoid unnecessary debugging overhead, and keep the console window open after a console app finishes so that you can read its output.

6. What method is used as the entry point when you do not use top-level statements?

   Answer: When top-level statements are not used, execution begins in a `Main` method, normally declared in a class such as `Program`. They can be explicitly declared:

   ```
   public static void Main() // Minimum
   public static int Main(string[] args) // Recommended

   public static async Task Main() // Asynchronous
   public static async Task<int> Main(string[] args) // Asynchronous
   ```

   With .NET 6 and later, the entry point method of a console app is implicitly declared by the compiler using the top-level program feature, as shown in the following code:

   ```
   public static int <Main>$(string[] args)
   public static async Task<int> <Main>$(string[] args)
   ```

7. What is the difference between a project-based C# app and a file-based C# app?

   Answer: A project-based C# app has a project file and can use the full project system, including NuGet packages, build configuration, multiple source files, resources, testing, and publishing options. A file-based C# app can run directly from a .cs file without first creating a project, making it useful for small programs, experiments, scripts, and demonstrations.

8. What is the difference between LTS and STS releases of .NET?

   Answer: Long-Term Support, or LTS, releases receive support for a longer period and are often preferred for applications that value stability and a longer maintenance window. Standard-Term Support, or STS, releases have a shorter support period but provide earlier access to newer .NET features.

9. Why can C#, F#, and Visual Basic all be used to create applications that run on .NET?

   Answer: C#, F#, and Visual Basic can all target .NET because their compilers translate source code into compatible Intermediate Language, or IL, together with metadata. The .NET runtime can then execute the compiled assemblies regardless of which supported .NET language produced them.

10. What information should you include when asking for help in a public programming forum or Discord channel?

    Answer: Include a clear description of what you are trying to achieve, the relevant source code, the exact error message and stack trace, the steps needed to reproduce the problem, and details about your environment, such as the operating system, .NET version, editor or IDE, and package versions. Explain what you expected to happen, what actually happened, and what you have already tried. Remove passwords, API keys, personal information, and other confidential data before posting.

Chapter 2 — Speaking C#

Exercise 2.3 — Test your knowledge

Suggested answers to these questions:

1. What is the difference between a .NET SDK version, a C# language version, and a target framework?

   Answer: A .NET SDK version identifies the installed tools used to create, build, test, and publish applications. A C# language version determines which C# syntax and language features the compiler accepts. A target framework, such as net11.0, determines which .NET APIs the application can use and which .NET runtime it is intended to run on.

2. Which directive can you temporarily add to a C# file to display the compiler and language versions as an error?

   Answer: Temporarily add the following directive:

   ```
   #error version
   ```

   The compiler reports an error that includes the compiler version and the selected C# language version.

3. What character normally marks the end of a C# statement?

   Answer: A semicolon `;` normally marks the end of a C# statement.

4. What is the difference between a line comment and a block comment?

   Answer: A line comment begins with `//` and continues to the end of the current line. A block comment begins with `/*` and ends with `*/`, so it can span part of a line or multiple lines.

5. What is the purpose of braces in C#?

   Answer: Braces `{ }` group statements into a block. They are used to define the bodies of constructs such as namespaces, types, methods, loops, and conditional statements.

6. What does a `using` directive allow you to omit when referring to a type?

   Answer: A `using` directive allows you to omit a type's namespace when referring to that type. For example, after importing `System`, you can write `Console` instead of `System.Console`.

7. What is the difference between a normal `using` directive and a `global using` directive?

   Answer: A normal `using` directive applies only to the source file in which it appears. A `global using` directive applies to every source file in the project.

8. Which naming convention is normally used for local variables, and which is normally used for types and methods?

   Answer: Local variables and parameters normally use camel case, such as `firstName`. Types, methods, properties, and other public members normally use Pascal case, such as `CustomerAccount` or `CalculateTotal`.

9. What is the difference between a `char` value and a `string` value?

   Answer: A `char` stores one UTF-16 code unit and is written using single quotation marks, such as `'A'`. A `string` stores a sequence of zero or more characters and is written using double quotation marks, such as `"Hello"`.

10. When would you use a verbatim string literal rather than an ordinary string literal?

    Answer: Use a verbatim string literal when you want backslashes and most other characters to be treated literally. It is especially useful for Windows file paths and multi-line text:

    ```
    string path = @"C:\Users\Mark\Documents";
    ```

    In a verbatim string, a double quotation mark is represented by two double quotation marks.

11. Why can adding `0.1` and `0.2` using `double` produce a result that is not exactly equal to `0.3`?

    Answer: The `double` type uses binary floating-point representation. Many decimal fractions, including `0.1` and `0.2`, cannot be represented exactly in binary, so calculations can contain tiny rounding errors. Therefore, `0.1 + 0.2` might not compare as exactly equal to `0.3`.

12. Which numeric type would normally be most suitable for money, and why?

    Answer: The `decimal` type is normally the most suitable numeric type for money because it represents base-10 decimal values with high precision. Values such as `0.1` can usually be represented exactly, reducing the rounding surprises associated with binary floating-point types.

13. How does a variable declared with `var` differ from one declared with `dynamic`?

    Answer: A variable declared with `var` is still statically typed. The compiler infers its type at compile time, and the type cannot later change:

    ```
    var age = 42; // Inferred as int.
    ```

    A variable declared with `dynamic` bypasses most compile-time type checking. Its operations are resolved at runtime, so errors that would normally be caught during compilation might instead cause runtime exceptions.

14. How can you determine how many bytes a type such as `double` uses in memory?

    Answer: Use the `sizeof` operator for supported value types:

    ```
    Console.WriteLine(sizeof(double)); // 8
    ```

    You can also use the type's static `Size` property when one is available, such as `IntPtr.Size`, or use APIs such as `Unsafe.SizeOf<T>()` for more advanced scenarios.

15. How do you right-align a format string?

    Answer: Use a positive alignment width in a composite format item or interpolated string. For example, the following right-aligns the value in a field ten characters wide:

    ```
    Console.WriteLine("{0,10}", 123);
    Console.WriteLine($"{123,10}");
    ```

    A negative alignment width left-aligns the value:

    ```
    Console.WriteLine($"{123,-10}");
    ```

Test your knowledge of number types

What type would you choose for the following "numbers"?

1. A person's telephone number

   Answer: `string`

2. A person's height

   Answer: `float`, `double`, `decimal`, or an integer number of millimeters

3. A person's age

   Answer: `int` for best performance on most CPUs or `byte` (0 to 255) for the smallest size, and it is usually better derived from a date of birth than stored as a number

4. A person's salary

   Answer: `decimal`, but currency and rounding policy matter

5. A book's ISBN

   Answer: `string`

6. A book's price

   Answer: `decimal`, but currency and rounding policy matter

7. A book's shipping weight

   Answer: `float` or `double`

8. A country's population

   Answer: `uint` (0 to about 4 billion), `int`, or `long`

9. The number of stars in the universe

   Answer: `ulong` (0 to about 18 quintillion) or `System.Numerics.BigInteger` (which allows an arbitrarily large integer), and may be an estimate represented in scientific notation

10. The number of employees in each of the small or medium businesses in the United Kingdom (up to about 50,000 employees per business)

    Answer: Since there are hundreds of thousands of small or medium businesses, we need to take memory size as the determining factor, so choose `ushort` because it only takes 2 bytes compared to `int`, which takes 4 bytes.

Chapter 3 — Controlling Flow, Converting Types, and Handling Exceptions

Exercise 3.2 — Practice exercises

Explore loops and overflow

What will happen if this code executes?

```
int max = 500;
for (byte i = 0; i < max; i++)
{
  WriteLine(i);
}
```

Answer:

The code will loop forever because the value of `i` can only be between `0` and `255`. Once `i` gets incremented beyond `255`, it loops back to `0` and, therefore, will always be less than `max` (`500`).

To prevent the infinite loop, you can add a checked statement around the code. This would cause an exception to be thrown after `255` due to the overflow, as shown in the following output:

```
254
255
System.OverflowException says Arithmetic operation resulted in an overflow.
```

A code solution can be found at the following link:

`https://github.com/markjprice/cs15net11/tree/main/code/Chapter03/Exercise_LoopsAndOverflow`.

Exercise 3.3 — Test your knowledge

Suggested answers to these questions:

1. What is the difference between a unary, binary, and ternary operator?

   Answer: A unary operator works with one operand, such as `!isValid` or `-number`. A binary operator works with two operands, such as `x + y` or `left == right`. A ternary operator works with three operands. C# has one ternary operator, the conditional operator:

   ```
   condition ? valueIfTrue : valueIfFalse
   ```

2. How do `x = y++;` and `x = ++y;` produce different results?

   Answer: In `x = y++;`, the postfix increment operator assigns the current value of `y` to `x` and then increments `y`. In `x = ++y;`, the prefix increment operator increments `y` first and then assigns the new value to `x`. For example, when `y` starts as `5`:

   ```
   x = y++; // x is 5, y is 6.
   x = ++y; // y is incremented first, so x and y receive the new value.
   ```

3. Why does `11 / 3` produce a different result from `11.0 / 3`?

   Answer: `11 / 3` performs integer division because both operands are integers. Its result is `3`, with the fractional part discarded. `11.0 / 3` performs floating-point division because `11.0` is a `double`, so its result is approximately `3.6666666667`.

4. What value does the remainder operator `%` calculate?

   Answer: The remainder operator, `%`, calculates the remainder left after division. For example, `11 % 3` is `2` because `3` fits into `11` three times, leaving `2`.

5. What is the difference between `&` and `&&` when their operands are Boolean values?

   Answer: When used with Boolean operands, `&` evaluates both operands regardless of the result of the first operand. `&&` uses short-circuit evaluation: if its left operand is `false`, the right operand is not evaluated because the whole expression must already be `false`.

6. What normally happens to a positive integer when it is shifted left by one bit?

   Answer: Shifting a positive integer left by one bit normally doubles its value. For example, binary `0011`, which represents `3`, becomes `0110`, which represents `6`. This rule can fail when significant bits are shifted out of the type's available range, causing overflow or a change of sign.

7. Why can guard clauses make a method easier to read, and when might an `else` still be appropriate?

   Answer: Guard clauses handle invalid inputs and exceptional cases near the beginning of a method, usually by returning or throwing an exception. This reduces nesting and leaves the method's main path easier to follow. An `else` can still be appropriate when two branches represent equally important alternatives or when keeping the alternatives together makes their relationship clearer.

8. Why must more specific patterns normally appear before more general patterns in a `switch`?

   Answer: A more general pattern can also match values handled by a more specific pattern. If the general pattern appears first, the specific pattern might never be reached. C# therefore reports an error for unreachable switch cases and normally requires specific patterns to appear before broader ones.

9. What does the discard pattern _ mean in a switch expression?

   Answer: The discard pattern, _, matches any value that has not already been matched by an earlier arm. It is often used as the final arm of a switch expression to provide a default result or throw an exception.

   ```
   string description = number switch
   {
       0 => "Zero",
       > 0 => "Positive",
       _ => "Negative"
   };
   ```

10. What is the difference between a `while` loop and a `do` loop?

    Answer: A `while` loop tests its condition before executing its body, so its body might not run at all. A `do` loop executes its body first and tests the condition afterward, so its body always runs at least once.

11. What are the three sections of a `for` statement, and which of them can be omitted?

    Answer: The three sections of a `for` statement are:

    ```
    for (initializer; condition; iterator)
    ```

    The initializer normally declares or initializes a loop variable. The condition determines whether the loop continues. The iterator normally updates the loop variable after each iteration. Any or all three sections can be omitted, although the two semicolons must remain:

    ```
    for (;;)
    {
        // An infinite loop unless exited using break, return, or an
    exception.
    }
    ```

12. What do `break` and `continue` do when used inside a loop?

    Answer: `break` immediately exits the nearest enclosing loop and continues execution after it. `continue` skips the remainder of the current iteration and begins the next iteration, first performing the iterator section in a `for` loop or retesting the condition in a `while` or `do` loop.

13. How does a rectangular multidimensional array differ from a jagged array?

    Answer: A rectangular multidimensional array has a fixed number of dimensions arranged in a rectangular shape. Every row therefore has the same number of elements, and elements are accessed using a comma-separated index:

    ```
    int[,] grid = new int[3, 4];
    int value = grid[1, 2];
    ```

    A jagged array is an array whose elements are themselves arrays. Each inner array can have a different length, and elements are accessed using separate pairs of brackets:

    ```
    int[][] rows = new int[3][];
    rows[0] = new int[2];
    rows[1] = new int[5];

    int value = rows[1][2];
    ```

14. What can happen when a `long` value is explicitly converted to an `int`?

    Answer: A `long` can contain values outside the range supported by `int`. When a `long` is explicitly converted to an `int`, information can be lost. In an unchecked context, the value normally wraps by retaining only the lower 32 bits. In a checked context, an `OverflowException` is thrown if the value is outside the `int` range.

    ```
    long large = 3_000_000_000;

    int wrapped = unchecked((int)large);
    int checkedValue = checked((int)large); // Throws OverflowException.
    ```

15. Why should specific exception types be caught before `Exception`, and why should an empty `catch` block be avoided?

    Answer: Catch blocks are tested from top to bottom. Because `Exception` is the general base class for most exceptions, placing it before a more specific exception type would make the specific catch block unreachable. Specific exceptions should therefore be caught first.

    An empty catch block silently ignores the exception. This can hide defects, leave data or program state inconsistent, and make problems difficult to diagnose. A catch block should normally recover from the problem, add useful context, log it, notify the caller, or rethrow the exception.

Test your knowledge of operators

A code solution can be found at the following link:

`https://github.com/markjprice/cs15net11/tree/main/code/Chapter03/Exercise_Operators`.

1. What are the values of `x` and `y` after the following statements execute?

```
x = 3;
y = 2 + ++x;
```

   Answer: `x` is `4` and `y` is `6`.

2. What are the values of `x` and `y` after the following statements execute?

```
x = 3 << 2;
y = 10 >> 1;
```

   Answer: `x` is `12` and `y` is `5`.

3. What are the values of `x` and `y` after the following statements execute?

```
x = 10 & 8;
y = 10 | 7;
```

   Answer: `x` is `8` and `y` is `15`.

Chapter 4 — Writing, Debugging, and Testing Functions

Exercise 4.3 — Test your knowledge

Suggested answers to these questions:

1. What does the `void` return type indicate about a function?

   Answer: The `void` return type indicates that a function does not return a value to its caller. The function can perform work, change state, write output, or throw an exception, but it cannot return a result using `return value;`.

2. What is the difference between a parameter and an argument?

   Answer: A parameter is a variable declared as part of a function's definition. An argument is the value or expression supplied for that parameter when the function is called.

```
static void Greet(string name) // name is a parameter.
{
    Console.WriteLine($"Hello, {name}!");
}

Greet("Alice"); // "Alice" is an argument.
```

3. When can named arguments make a function call easier to understand?

   Answer: Named arguments can make a function call easier to understand when a method has several parameters of the same type, when the meaning of a literal value is unclear, or when optional arguments are being supplied out of their normal order.

   ```
   ConfigureConnection(timeout: 30, useEncryption: true);
   ```

4. What makes a parameter optional, and what happens when its argument is omitted?

   Answer: A parameter is optional when its declaration provides a default value.

   ```
   static void Greet(string name = "Guest")
   ```

   When the caller omits the argument, the compiler uses the parameter's default value.

5. What is the difference between a local function in a top-level program and a static method in a partial `Program` class?

   Answer: A local function declared in a top-level program is local to the compiler-generated entry-point method. It can access variables declared in the surrounding top-level code, but it cannot normally be called from other classes.

   A static method declared in a partial `Program` class is a regular member of the generated `Program` type. It cannot capture top-level local variables, but it can be given accessibility modifiers and called from other code when its accessibility permits.

6. Why must an explicitly declared partial `Program` class use the same namespace as the compiler-generated `Program` class?

   Answer: The `partial` keyword combines declarations only when they represent the same type. A type's namespace is part of its fully qualified identity, so a partial `Program` class declared in a different namespace would be a separate type rather than another part of the compiler-generated Program class.

7. What information should useful XML documentation provide beyond repeating the method signature?

   Answer: Useful XML documentation should explain what the method does, the meaning and valid range of each parameter, what the method returns, any side effects, and the exceptions callers should expect. It can also describe special cases, assumptions, and examples that are not obvious from the signature alone.

8. What is the purpose of the base case in a recursive function?

   Answer: The base case defines the condition under which a recursive function stops calling itself. Without a reachable base case, recursion continues until the program exhausts the available stack and usually throws a `StackOverflowException`.

9. What does a `checked` block do when an integer arithmetic operation overflows?

   Answer: A `checked` block causes an `OverflowException` to be thrown when a checked integer arithmetic operation or numeric conversion produces a value outside the destination type's supported range:

   ```
   int max = int.MaxValue
   checked
   {
       int result = max + 1; // Throws OverflowException.
   }
   ```

10. In VS Code or Visual Studio, what is the difference between pressing *F5*, *Ctrl* or *Cmd* + *F5*, *Shift* + *F5*, and *Ctrl* or *Cmd* + *Shift* + *F5*?

    Answer: The keyboard shortcuts normally perform these actions:

    - *F5* starts debugging or continues execution to the next breakpoint.
    - *Ctrl* + *F5* on Windows and Linux, or *Cmd* + *F5* on macOS, starts the program without attaching the debugger.
    - *Shift* + *F5* stops the current debugging session.
    - *Ctrl* + *Shift* + *F5* on Windows and Linux, or *Cmd* + *Shift* + *F5* on macOS, restarts the debugging session.

    The exact shortcut can vary if the editor's keyboard mappings have been customized.

11. When debugging a method call, what is the difference between Step Into and Step Over?

    Answer: **Step Into** enters the called method so that you can debug its statements one at a time. **Step Over** executes the entire method call without entering it, then pauses at the next statement in the current method.

12. When writing a unit test, what are the three "A"s?

    Answer: The three "A"s of a unit test are:

    - Arrange: Set up the objects, inputs, and expected results.
    - Act: Call the code being tested.
    - Assert: Verify that the actual result matches the expected result.

13. What is the difference between an xUnit `[Fact]` and an xUnit `[Theory]` using `[InlineData]`?

    Answer: An xUnit `[Fact]` represents one test with no externally supplied test data. An xUnit `[Theory]` is a parameterized test that can run multiple times with different inputs. Each `[InlineData]` attribute supplies one set of arguments for one test run.

    ```
    [Fact]
    public void TwoPlusTwoEqualsFour()
    {
        Assert.Equal(4, 2 + 2);
    ```

```
}

[Theory]
[InlineData(2, 3, 5)]
[InlineData(-1, 1, 0)]
public void AddReturnsExpectedResult(int a, int b, int expected)
{
    Assert.Equal(expected, a + b);
}
```

14. Which `dotnet` command builds and runs the tests in a test project or solution?

    Answer: Use the following command:

    ```
    dotnet test
    ```

    It restores dependencies when necessary, builds the test project or solution, discovers the tests, runs them, and reports the results.

15. What statement should you use to rethrow a caught exception named `ex` without losing the stack trace?

    Answer: Use `throw;` to rethrow the currently caught exception while preserving its original stack trace:

    ```
    catch (Exception ex)
    {
        Console.WriteLine(ex.Message);
        throw;
    }
    ```

    Do not use `throw ex;`, because that resets the stack trace to the current location and can hide where the exception originally occurred.

Chapter 5 — Building Your Own Types with Object-Oriented Programming

Exercise 5.2 — Practice with access modifiers

The code given in this exercise will cause the compiler to show the following errors:

```
Error CS0122 'Car' is inaccessible due to its protection level
Error CS0122 'Car.Start()' is inaccessible due to its protection level
Error CS0122 'Car.Wheels' is inaccessible due to its protection level
```

In the class library project, in `Car.cs`, you would need to apply the `public` access modifier to the class, as the default access modifier for a type is `internal`, and also to the `Wheels` property and the `Start` method, as the default access modifier for a member is `private`, as shown highlighted in the following code:

```
public class Car
{
  public int Wheels { get; set; }
  public bool IsEV { get; set; }
  public void Start()
  {
    Console.WriteLine("Starting...");
  }
}
```

Exercise 5.3 – Test your knowledge

Suggested answers to these questions:

1. What is the difference between a **type** and an **object** created from that type?

   Answer: A **type** is a definition that describes the data and behavior that its values or objects can have. An **object** is a particular instance created from a reference type. For example, `Person` can be a type, while `new Person()` creates an object of that type. Multiple variables can refer to the same object.

2. Why might reusable types be placed in a class library instead of directly in a console app?

   Answer: Reusable types can be placed in a class library so that they are separated from the application's user interface and startup code. The library can then be referenced by multiple applications, services, or test projects without duplicating the source code. This separation also makes the types easier to maintain and test independently.

3. What are the default access modifiers for a top-level class and for a class member?

   Answer: A top-level class declared without an access modifier is `internal`, so it is accessible only within its assembly. A class member declared without an access modifier is `private`, so it is accessible only inside its containing class.

4. How do instance, `static`, `const`, and `readonly` members differ?

   Answer: An **instance member** belongs to an individual object, so different objects can have different values. A **static member** belongs to the type itself and is shared by all its objects. A `const` member is a compile-time constant, is implicitly static, and must be initialized with a constant expression. A `readonly` field can be assigned at its declaration or in a constructor, but it cannot be reassigned afterward. A `readonly` field can be either an instance field or a `static readonly` field.

5. Why should the individual values in a flags `enum` normally be powers of two?

   Answer: The individual values in a flags enum should normally be powers of two, such as `1`, `2`, `4`, and `8`, because each value then occupies a separate bit. This allows values to be combined with the bitwise OR operator without one flag being confused with a combination of other flags.

   ```
   [Flags]
   enum FilePermissions
   {
       None = 0,
       Read = 1,
       Write = 2,
       Execute = 4
   }
   ```

   For example, `Read | Write` produces `3`, which has both the `Read` and `Write` bits set.

6. How does a `required` member differ from a non-nullable member?

   Answer: A `required` member must be assigned when an object is created, either by a constructor recognized as initializing required members or by an object initializer. A non-nullable member expresses that the member should not contain `null`, but it does not necessarily require the caller to set it in an object initializer.

   The two features enforce different rules. A required nullable property must be assigned, but it can be assigned `null`. A non-nullable property might not be marked `required` if it is initialized by its declaration or constructor.

7. What does a constructor do, and when is it called?

   Answer: A constructor initializes a newly created object. It normally assigns initial values to fields and properties and establishes any conditions the object needs to be valid. An instance constructor is called when an object is created with `new`, before the new object is returned to the caller.

   ```
   Person person = new Person("Alice");
   ```

   Here, the `Person` constructor is called as part of creating the object.

8. What forms part of a method's signature for overload resolution?

   Answer: For overload resolution, a method's signature includes its name, the number of generic type parameters, and the ordered types and parameter-passing modes of its parameters. Parameter names and optional default values do not distinguish overloads, and the return type is not part of the signature for overload resolution.

9. Why can two methods not be overloaded by changing only their return types?

   Answer: Two methods cannot be overloaded by changing only their return types because the compiler chooses an overload from the method name and the arguments at the call site. The return value might be ignored or used in a context that does not provide enough information to distinguish the methods.

   For example, these declarations are not permitted:

   ```
   static int GetValue() => 42;
   static string GetValue() => "Forty-two";
   ```

   A call such as `GetValue();` would provide no way to choose between them.

10. Where must optional and `params` parameters appear in a parameter list?

    Answer: Optional parameters must appear after all required parameters. A `params` parameter must be the final parameter, and a parameter list can contain only one `params` parameter.

11. What are the differences between passing an argument by value, `in`, `ref`, and `out`?

    Answer: Passing an argument **by value** copies its value into the parameter. Changes to the parameter itself do not change the caller's variable.

    An `in` parameter is passed by reference but is read-only inside the method. A `ref` parameter is passed by reference and can be read and reassigned by the method; the caller's variable must already be initialized. An `out` parameter is passed by reference for output; the method must assign it before returning, and the caller does not have to initialize it first.

12. What happens when a reference-type variable is passed to a method by value?

    Answer: When a reference-type variable is passed by value, the reference is copied, not the object. The method and caller therefore refer to the same object, so the method can modify that object. However, assigning a different object or `null` to the parameter does not change the caller's variable.

    ```
    static void ChangePerson(Person person)
    {
        person.Name = "Changed";       // Changes the shared object.
        person = new Person("Other");  // Does not change the caller's
    variable.
    }
    ```

    To allow the method to replace the object referenced by the caller's variable, the parameter would need to be declared with `ref`.

13. When would a tuple be more appropriate than defining a new class or record?

    Answer: A tuple is appropriate for a small, short-lived grouping of values, especially when returning several related results from a method or holding temporary intermediate data:

```
(string Name, int Age) GetPersonDetails()
{
    return ("Alice", 42);
}
```

A class, record, or struct is usually more appropriate when the concept is used widely, appears in a public API, needs validation or behavior, or has lasting meaning in the application's domain.

14. What is the difference between a field, a property, and an indexer?

    Answer: A **field** is a storage location declared directly in a type. A **property** provides accessor methods that control, validate, calculate, or expose a value while being used with field-like syntax. An **indexer** resembles a property but accepts one or more arguments, allowing an object to be accessed with array-like syntax:

    ```
    public string nameField;              // Field
    public string Name { get; set; }      // Property
    public string this[int index] =>      // Indexer
        names[index];
    ```

    Properties do not necessarily store values because their getters can calculate a result. Indexers are accessed through expressions such as `objectName[2]`.

15. How do classes, records, and union types differ in their treatment of identity, equality, and valid states?

    Answer: A **class** normally models an entity with identity. Two separate class objects are not equal by default merely because their fields contain the same values; ordinary classes use reference equality unless they override that behavior.

    A **record** normally models a value. Records provide value-based equality, so two records of the same type are equal when their corresponding values are equal.

    A **union type** states that a value must be exactly one of a fixed set of case types. Its main purpose is to constrain the valid states and allow the compiler to check that every case is handled. A union does not itself add record-style equality, cloning, or deconstruction behavior; those behaviors depend on its case types.

16. Why is the `partial` keyword useful?

    Answer: The `partial` keyword allows one type or member definition to be divided across multiple source files. The compiler combines the parts into a single type. This is useful for separating generated code from handwritten code, dividing a large type into logical sections, and allowing source generators or visual designers to provide part of an implementation without overwriting the developer's code.

17. What is the difference between the following two statements? (Do not just say a > character!)

```
public List<Person> Children = new();
public List<Person> Children => new();
```

Answer: The first statement declares a field and initializes it with one list when the `Person` object is created:

```
public List<Person> Children = new();
```

Every access to `Children` on that object returns the same list. Items added to it remain in the list. Because the field is public and not `readonly`, callers can also replace the entire list.

The second statement declares an expression-bodied, get-only property:

```
public List<Person> Children => new();
```

Its getter executes `new()` every time the property is accessed, so every access returns a different empty list. An item added through one access is immediately lost when that list is no longer referenced:

```
person.Children.Add(child);
Console.WriteLine(person.Children.Count); // 0
```

A suitable get-only property that retains its list would be:

```
public List<Person> Children { get; } = new();
```

Properties can compute and return a value each time their getter runs; an expression-bodied property is not automatically backed by persistent storage.

18. Why can't a `DateTime` field be a constant?

Answer: A `DateTime` field cannot be a constant because `DateTime` is a struct that is not one of C#'s permitted compile-time constant types. Constants are limited to simple numeric types, `bool`, `char`, `string`, enum types, and null references.

Use a `static readonly` field when the value must be created at runtime:

```
public static readonly DateTime LaunchDate =
    new(year: 2026, month: 11, day: 10);
```

Unlike a constant, a `static readonly` field can be initialized by calling a constructor.

Chapter 6 — Implementing Interfaces and Inheriting Classes

Exercise 6.3 — Test your knowledge

Suggested answers to these questions:

1. How does an instance method differ from a static method?

   Answer: An **instance method** belongs to a particular object and can access that object's instance members through `this`. A **static method** belongs to the type itself, is normally called using the type name, and cannot directly access instance members because it has no current object.

2. What conditions must an operator declaration satisfy, and when is operator overloading appropriate?

   Answer: An operator declaration must be `public`. A unary operator has one operand, and a binary operator has two. At least one operand must be the containing type, or its nullable form. Most operator overloads are `static`, although C# 14 also allows instance implementations of compound-assignment operators and the increment and decrement operators. Some operators must be declared in pairs, including `==` and `!=`, `<` and `>`, `<=` and `>=`, and `true` and `false`.

   Operator overloading is appropriate when an operator has an obvious and unsurprising meaning for the type, such as adding two vectors, comparing two measurements, or multiplying a matrix. It should not be used merely to shorten a method call when the operator's meaning would be unclear.

3. Why is `Dictionary<TKey, TValue>` safer than `Hashtable`?

   Answer: `Dictionary<TKey, TValue>` is safer than `Hashtable` because it is generic and therefore enforces its key and value types at compile time. A `Hashtable` stores keys and values as `object`, so values often require casts when retrieved, and an incorrect cast might fail at runtime. Generic collections also avoid much of the boxing and unboxing required when value types are stored in a `Hashtable`. For greenfield developments, `Hashtable` should be avoided in favor of `Dictionary<TKey, TValue>`: `https://learn.microsoft.com/en-us/dotnet/api/system.collections.hashtable?view=net-11.0#remarks`.

4. What do the type parameters `TKey` and `TValue` represent?

   Answer: `TKey` represents the type of each key in the dictionary, and `TValue` represents the type of the value associated with each key:

   ```
   Dictionary<string, int> ages = new();
   ```

   In this example, `TKey` is `string` and `TValue` is `int`.

5. What promise does a type make when it implements an interface?

   Answer: When a type implements an interface, it promises to provide the behavior specified by that interface's contract. Code using an interface variable can therefore call the interface's members without needing to know the object's concrete type. Members with default implementations are an exception because the interface has already supplied their behavior.

6. What is the difference between implicit and explicit interface implementation?

   Answer: An **implicit interface implementation** is normally a public member whose name and signature match the interface member. It can be called through either the concrete type or the interface type.

   ```
   public void Draw()
   {
   }
   ```

   An **explicit interface implementation** includes the interface name and does not have an access modifier:

   ```
   void IDrawable.Draw()
   {
   }
   ```

   It can be called only through a variable or cast of the interface type. Explicit implementation is useful when two interfaces contain conflicting member names or when an interface member should not appear as part of the class's ordinary public API.

7. What problem can a default interface implementation solve?

   Answer: A default interface implementation allows a new interface member to be added with existing behavior without immediately requiring every implementing type to add its own implementation. It can therefore help an interface evolve without breaking all existing implementations. Implementing types can use the default behavior or replace it.

8. How does `int?` differ from `string?`?

   Answer: `int?` and `string?` use similar syntax but work differently.

   `int?` is shorthand for `Nullable<int>`. It is a different value type that can contain either an `int` value or no value:

   ```
   int? score = null;
   ```

   `string?` is a nullable-reference-type annotation. It tells the compiler that the reference is intended to be allowed to contain `null`. It does not create a different runtime type from `string`; nullable reference types primarily affect compile-time analysis and warnings.

9. What are the respective purposes of the `?.`, `??`, and `!` operators?

   Answer: The operators have different purposes:

   - `?.` performs null-conditional member access. It accesses the member only when the receiver is not `null`; otherwise, the expression produces `null`.
   - `??` is the null-coalescing operator and it provides a fallback value when its left operand is `null`.
   - `!` is the null-forgiving operator. It suppresses a nullable warning by telling the compiler to treat an expression as not null. It performs no runtime check and does not prevent a `NullReferenceException`.

   ```
   int? length = name?.Length;
   string displayName = name ?? "Unknown";
   Console.WriteLine(name!.Length);
   ```

10. What is the difference between hiding a member with `new` and overriding it with `virtual` and `override`?

    Answer: The `new` keyword **hides** an inherited member. Which hidden member is selected normally depends on the compile-time type of the variable through which it is called.

    The `virtual` and `override` keywords provide polymorphic behavior. A base member is marked `virtual`, and a derived class replaces its behavior with `override`. Which override runs depends on the runtime type of the object.

11. How do `abstract`, `sealed`, and `closed` affect a class hierarchy?

    Answer: An `abstract` class cannot be instantiated and is intended to serve as a base class. It can declare abstract members that derived concrete classes must implement.

    A `sealed` class cannot be used as a base class, so no further classes can derive from it.

    A `closed` class defines a restricted hierarchy. It is implicitly abstract, and its direct derived classes must be declared in the same assembly. Because the compiler knows all direct descendants, it can recognize a `switch` expression that handles all of them as exhaustive. A class derived from a closed class is not itself automatically closed, so other assemblies might still derive from that descendant unless it is also `closed` or `sealed`.

12. When a base-class variable refers to a derived-class object, how is an overridden method selected?

    Answer: When a base-class variable refers to a derived-class object, a call to an overridden virtual method is selected according to the object's runtime type. The most-derived applicable override runs, even though the variable itself is declared using the base type.

    ```
    Animal animal = new Cat();
    animal.MakeSound(); // Calls Cat.MakeSound if it overrides the method.
    ```

    This behavior is runtime polymorphism.

13. How can a declaration pattern safely test and cast an object in one expression?

    Answer: A declaration pattern uses `is` to test the runtime type and declare a correctly typed variable in one expression:

    ```
    if (value is Person person)
    {
        Console.WriteLine(person.Name);
    }
    ```

    The variable `person` is assigned only when `value` is non-null and compatible with `Person`, so no separate unsafe cast is needed.

14. Why should a custom exception normally provide constructors accepting no arguments, a message, and a message with an inner exception?

    Answer: A custom exception should normally provide the three common constructors so callers can create it in the same standard ways as built-in exceptions.

    The parameterless constructor supports an exception with a default message. The message constructor allows the caller to describe the problem. The message-and-inner-exception constructor preserves the original exception when the custom exception adds higher-level context.

15. How is an extension method defined, and what happens if a matching instance method already exists?

    Answer: Using the traditional syntax, an extension method is defined as a static method in a non-nested, nongeneric static class. Its first parameter uses the `this` modifier to identify the extended type:

    ```
    public static class StringExtensions
    {
        public static int WordCount(this string value)
        {
            return value.Split(
                ' ',
                StringSplitOptions.RemoveEmptyEntries).Length;
        }
    }
    ```

    It can then be called using instance-member syntax:

    ```
    int count = text.WordCount();
    ```

    C# also supports declaring extension members in an extension block. If an applicable instance method already exists, the instance method takes precedence over an extension method with the same call syntax.

16. What is the difference between the `is` and `as` operators?

    Answer: The `is` operator tests whether a value is compatible with a type or matches a pattern. It returns `true` or `false`, and a declaration pattern can also create a typed variable:

    ```
    if (value is Customer customer)
    {
    }
    ```

    The `as` operator attempts a conversion. If the conversion succeeds, it returns the converted reference; if it fails, it returns `null` instead of throwing `InvalidCastException`:

    ```
    Customer? customer = value as Customer;
    ```

    The `as` operator is limited to conversions for which `null` can represent failure, such as reference types and nullable value types.

17. Which keyword is used to prevent a class from being derived from or a method from being further overridden?

    Answer: Use the `sealed` keyword to prevent a class from being derived from:

    ```
    public sealed class FinalClass
    {
    }
    ```

    It can also prevent a particular overridden member from being overridden again:

    ```
    public sealed override void Run()
    {
    }
    ```

    A non-virtual method cannot be marked as sealed.

18. Which keyword is used to prevent a class from being instantiated with the `new` keyword?

    Answer: Use the `abstract` keyword to prevent a class from being instantiated directly with `new`:

    ```
    public abstract class Shape
    {
    }
    ```

    A non-abstract class must derive from it before objects can be created.

19. What's the difference between a destructor and a `Deconstruct` method?

    Answer: What C# traditionally calls a **destructor** is more accurately called a finalizer:

    ```
    ~ResourceHolder()
    {
    }
    ```

It is invoked by the garbage collector before reclaiming an object that requires finalization. Its execution time is nondeterministic, so it should not be treated like a C++ destructor. Deterministic resource cleanup should normally use `IDisposable` and `Dispose`.

A `Deconstruct` method is an ordinary method that separates an object into component values using `out` parameters:

```
public void Deconstruct(
    out string name,
    out int age)
{
    name = Name;
    age = Age;
}
```

It enables deconstruction syntax:

```
(string name, int age) = person;
```

A finalizer participates in object cleanup; a `Deconstruct` method extracts values from an object.

20. What are the signatures of the constructors that all exceptions should have?

Answer: A custom exception should normally provide these three constructors:

```
public class InvalidPersonException : Exception
{
    public InvalidPersonException()
    {
    }

    public InvalidPersonException(string? message)
        : base(message)
    {
    }

    public InvalidPersonException(
        string? message,
        Exception? innerException)
        : base(message, innerException)
    {
    }
}
```

In signature form, they are:

```
InvalidPersonException()
InvalidPersonException(string? message)
InvalidPersonException(string? message, Exception? innerException)
```

The exact exception-class name changes for each custom exception, but the parameter patterns remain the same.

Chapter 7 — Packaging and Distributing .NET Types

Exercise 7.3 — Test your knowledge

Suggested answers to these questions:

1. What is the difference between a namespace and an assembly?

   Answer: A **namespace** is a logical naming system used to organize types and prevent name collisions. An **assembly** is a compiled deployment and versioning unit, normally stored as a `.dll` or `.exe`, that contains CIL, metadata, types, and possibly resources.

2. What roles do a language compiler, the CLR, and the BCL each perform?

   Answer: A **language compiler** checks source code written in a language such as C#, then normally translates it into Common Intermediate Language, or CIL, together with metadata describing its types and members. The **Common Language Runtime (CLR)**, loads and executes managed code, usually using a just-in-time compiler to translate CIL into native machine code. It also provides services such as garbage collection, exception handling, type safety, and thread management. The **Base Class Library (BCL)**, provides reusable types and APIs for common tasks such as working with strings, collections, files, networking, and dates and times.

3. Can one assembly contain multiple namespaces, and can one namespace span multiple assemblies?

   Answer: Yes. One assembly can contain types from multiple namespaces, and one namespace can contain types contributed by multiple assemblies. A namespace does not determine where a type is physically compiled or deployed.

4. What does a .NET project SDK provide to an SDK-style project?

   Answer: A .NET project SDK provides the MSBuild targets, tasks, properties, and default items needed to build a particular kind of project. For example, `Microsoft.NET.Sdk` supplies the normal rules for compiling, packing, and publishing .NET code and automatically includes source files that follow the expected conventions.

5. What is the difference between a `PropertyGroup` and an `ItemGroup`?

   Answer: A `PropertyGroup` contains named properties that normally have one value, such as the target framework, nullable setting, or output type:

   ```
   <PropertyGroup>
     <TargetFramework>net11.0</TargetFramework>
     <Nullable>enable</Nullable>
   </PropertyGroup>
   ```

   An `ItemGroup` contains collections of items, such as package references, project references, files, or resources. Items can also have associated metadata:

   ```
   <ItemGroup>
     <PackageReference Include="Humanizer" Version="2.14.1" />
   </ItemGroup>
   ```

   Properties are name-value pairs, whereas items normally represent collections of build inputs or dependencies.

6. What is the relationship between the C# `string` keyword and `System.String`?

   Answer: The C# keyword `string` is an alias for the .NET type `System.String`. They refer to exactly the same runtime type and compile to equivalent CIL:

   ```
   string first = "Hello";
   System.String second = "Hello";
   ```

   The keyword form does not require a `using System;` directive.

7. When should a reusable library target `netstandard2.0` rather than only `net11.0`?

   Answer: A reusable library should target `netstandard2.0` when it must be usable by older .NET implementations, particularly both .NET Framework and modern .NET. If the library is intended only for .NET 11 applications, targeting `net11.0` gives it access to newer APIs and runtime features.

   A widely distributed library might multi-target both:

   ```
   <TargetFrameworks>netstandard2.0;net11.0</TargetFrameworks>
   ```

   This provides broad compatibility through the .NET Standard build and newer capabilities through the .NET 11 build.

8. What information is represented by a runtime identifier such as `win-x64`?

   Answer: A **runtime identifier (RID)**, identifies the operating-system and processor-architecture environment for which an application or package asset is intended. In `win-x64`, `win` represents Windows and `x64` represents the 64-bit x86 processor architecture. RIDs allow .NET and NuGet to select platform-specific executables and native dependencies.

9. What is the difference between framework-dependent and self-contained deployment?

   Answer: A **framework-dependent** deployment contains the application and its dependencies but expects a compatible .NET runtime to be installed on the target computer. It is normally smaller and automatically uses an installed runtime containing the latest applicable servicing patches.

   A **self-contained** deployment includes the application and the required .NET runtime. The target computer does not need a separately installed .NET runtime, but the deployment is larger and must be published separately for each required platform.

10. Is single-file publishing necessarily self-contained? Explain your answer.

    Answer: No. Single-file publishing and self-contained deployment are separate choices. A single-file application can be either framework-dependent or self-contained:

    ```
    dotnet publish -r linux-x64 \
      -p:PublishSingleFile=true \
      --self-contained false
    ```

    This command creates a framework-dependent single-file application, so the target computer must still have the appropriate .NET runtime installed.

11. Why must the publisher of a self-contained application plan for .NET runtime servicing?

    Answer: A self-contained application carries its own copy of the .NET runtime and does not automatically roll forward to a newer runtime security patch installed on the computer. When Microsoft releases a runtime fix, the application publisher must rebuild, republish, and redistribute the application with the patched runtime.

12. Why can trimming cause code that uses reflection to fail?

    Answer: Trimming uses static analysis to remove code that appears not to be used. Reflection can locate types and members dynamically using names or values that are not known until runtime. The trimmer might therefore conclude that a reflected member is unused and remove it, causing the reflection operation to fail in the published application.

    Trim-compatible code can sometimes use annotations such as `DynamicallyAccessedMembers`, or replace reflection with source-generated code.

13. Name two features that are incompatible with or restricted by Native AOT.

    Answer: Features that are incompatible with or restricted by Native AOT include:

    - Loading assemblies dynamically, such as with `Assembly.LoadFile`.
    - Generating code at runtime with `System.Reflection.Emit`.

    Other restrictions include no C++/CLI support, no built-in COM support on Windows, required trimming, and limitations in some debugging, profiling, and ASP.NET Core features.

14. What is the difference between `dotnet pack` and `dotnet publish`, and how would you request an exact NuGet package version?

    Answer: `dotnet pack` creates a NuGet package, normally a `.nupkg` file, for other developers or projects to consume. dotnet publish produces application files that are ready to deploy and run on a target computer or hosting environment.

    To request an exact package version, set the `PackageVersion` MSBuild property:

    ```
    dotnet pack -p:PackageVersion=2.1.0
    ```

    With .NET 10 or later, `dotnet pack` also supports the following option:

    ```
    dotnet pack --version 2.1.0
    ```

15. How can you control the sources that a build process uses to download NuGet packages?

    Answer: NuGet package sources can be controlled using a repository-level `nuget.config` file:

    ```
    <?xml version="1.0" encoding="utf-8"?>
    <configuration>
      <packageSources>
        <clear />
        <add key="nuget.org"
             value="https://api.nuget.org/v3/index.json" />
      </packageSources>
    </configuration>
    ```

    A controlled build process can explicitly restore using that file and then build without restoring again:

    ```
    dotnet restore --configfile nuget.config
    dotnet build --no-restore
    ```

    Individual sources can also be supplied to `dotnet restore` using `--source`.

16. What problem does Central Package Management solve, and which file normally contains the centrally managed package versions?

    Answer: Central Package Management (CPM), prevents package versions from being repeated and potentially becoming inconsistent across many project files. It allows package versions for a repository or group of projects to be managed in one place, normally a file named: `Directory.Packages.props` that is usually placed at the root of the repository or solution. CPM is enabled by setting `ManagePackageVersionsCentrally` to `true` in that file.

17. When CPM is enabled, where is a package version declared, and what attribute is normally omitted from the corresponding `PackageReference`?

Answer: With CPM enabled, the package version is normally declared using a `PackageVersion` item in `Directory.Packages.props`:

```
<ItemGroup>
  <PackageVersion Include="Humanizer"
                  Version="2.14.1" />
</ItemGroup>
```

The project file then contains a corresponding `PackageReference` without a `Version` attribute:

```
<ItemGroup>
  <PackageReference Include="Humanizer" />
</ItemGroup>
```

NuGet obtains the version from the matching central `PackageVersion` item.

18. Why is Package Source Mapping useful when a project uses more than one NuGet package source?

    Answer: When several NuGet sources are configured, NuGet normally searches all of them for a requested package. The same package ID might exist on both a trusted private feed and a public feed, making the selected source unpredictable and creating a dependency-confusion risk.

    Package Source Mapping (PSM), restricts particular package IDs or ID patterns to specified sources. This makes restores more predictable and improves software-supply-chain security.

19. After PSM is enabled, which direct and transitive packages must match a package mapping pattern?

    Answer: After Package Source Mapping is enabled, every package ID used by the solution must match an applicable mapping pattern. This includes both direct packages listed in project files and transitive packages brought in as dependencies of other packages. If a package does not match a mapping, restore fails rather than searching an unmapped source.

20. What does `<clear />` do inside the `<packageSources>` element of a `nuget.config` file?

    Answer: A `<clear />` element inside `<packageSources>` removes package sources inherited from earlier configuration files, such as computer-level or user-level `nuget.config` files. Only sources added after `<clear />` in the effective configuration remain available:

    ```
    <packageSources>
      <clear />
      <add key="nuget.org"
           value="https://api.nuget.org/v3/index.json" />
    </packageSources>
    ```

    This improves repeatability by preventing an individual developer's or build computer's additional package sources from silently affecting restore behavior.

Chapter 8 — Working with Common .NET Types

Exercise 8.3 — Test your knowledge

Suggested answers to these questions:

1. Why would you use `BigInteger` instead of `ulong`?

   Answer: Use `BigInteger` when an integer might exceed the fixed range of `ulong`, whose maximum value is 18,446,744,073,709,551,615, or when arbitrarily large positive or negative integers are required. `BigInteger` expands as necessary, limited mainly by available memory, although its operations generally require more memory and processing than fixed-size integer types.

2. Why can `int * int` overflow even when you assign the result to a `long`?

   Answer: The types of the operands determine the type used to perform an arithmetic operation. If both operands are `int`, the multiplication is performed as an `int`; only afterward is its result converted to `long`. The overflow might therefore occur before the assignment:

   ```
   int a = 1_000_000;
   int b = 1_000_000;

   long incorrect = a * b;
   ```

   Convert at least one operand to `long` before multiplying:

   ```
   long correct = (long)a * b;
   ```

   The binary numeric-promotion rules select a common operand type, which is also normally the operation's result type.

   You would typically also wrap the multiplication in a `checked` block to detect overflow.

3. What values can be returned by `Random.Shared.NextInteger<int>(1, 7)`?

   Answer: `Random.Shared.NextInteger<int>(1, 7)` can return 1, 2, 3, 4, 5, or 6. The minimum is inclusive and the maximum is exclusive, making this call suitable for simulating a six-sided die.

4. Why should `System.Random` not be used to generate passwords or security tokens?

   Answer: `System.Random` is a pseudorandom-number generator designed for statistical and simulation purposes, not for resisting deliberate prediction. Its output must not be used for passwords, security tokens, cryptographic keys, or similar security-sensitive values. Use the static methods of `RandomNumberGenerator` from `System.Security.Cryptography` instead.

5. What is the difference between `Random.GetItems` and `Random.Shuffle`?

   Answer: `Random.GetItems` creates or fills a destination by repeatedly selecting random items from a set of choices. The same choice can be selected more than once, so duplicates are possible.

`Random.Shuffle` rearranges the existing elements of an array or span in place. It neither adds nor removes elements, so the shuffled collection contains exactly the same items as before, but usually in a different order.

6. What is the main difference between a version 4 GUID and a version 7 GUID?

   Answer: A version 4 GUID is primarily generated from random data. A version 7 GUID combines a Unix timestamp with random data, giving version 7 values a natural chronological sort order while retaining a very low probability of duplication.

   Version 7 GUIDs can therefore be preferable for database keys because newly generated values tend to be ordered by creation time rather than being distributed randomly throughout an index.

7. What does the `Length` property of a .NET `string` count?

   Answer: A string's `Length` property counts UTF-16 code units, which are represented by `char` values. It does not necessarily count Unicode scalar values or user-perceived characters. A supplementary Unicode character represented by a surrogate pair contributes `2` to `Length`, and a grapheme such as a letter followed by a combining accent can also contain multiple code units.

8. When should you use `Rune` instead of `char`?

   Answer: Use `Rune` when code must process complete Unicode scalar values rather than individual UTF-16 code units. A `char` can represent one Basic Multilingual Plane value or one half of a surrogate pair, whereas a `Rune` can represent any valid Unicode scalar value without splitting a surrogate pair.

   A `Rune` still does not necessarily represent one user-perceived character. Emoji sequences, flags, and letters with combining marks can contain multiple runes.

9. When should string comparison use ordinal rules, and when should it use culture-sensitive rules?

   Answer: Use ordinal comparison for non-linguistic values whose meaning should not vary by language or culture, such as protocol tokens, identifiers, command names, file extensions, and security-sensitive comparisons. Use `StringComparison.Ordinal` or `StringComparison.OrdinalIgnoreCase` as appropriate.

   Use culture-sensitive comparison for text presented to or entered by people when linguistic rules should apply, such as sorting names or searching natural-language text. This normally means `CurrentCulture` or `CurrentCultureIgnoreCase` for the current user's culture.

10. When is `StringBuilder` preferable to an interpolated string or a single concatenation expression?

    Answer: `StringBuilder` is preferable when constructing a string incrementally through many append operations, particularly inside a loop or when the number of additions is not known in advance. Because strings are immutable, repeatedly concatenating onto an existing string can create many temporary string objects.

For one interpolated string or a single expression joining a fixed, small number of values, interpolation or concatenation is normally simpler and does not justify creating a `StringBuilder`.

11. What does the regular expression `^\d+$` match?

    Answer: The regular expression `^\d+$` matches a nonempty input consisting entirely of decimal digit characters:

    - `^` anchors the match at the beginning.
    - `\d` matches a decimal digit.
    - `+` requires one or more digits.
    - `$` anchors the match at the end, or immediately before a final newline under the default .NET rules.

    In .NET, `\d` includes Unicode decimal digits from many writing systems, not only ASCII digits `0` through `9`. Use `[0-9]` when only ASCII digits should be accepted.

12. What benefit does `[GeneratedRegex]` provide over repeatedly constructing a `Regex` from a pattern string?

    Answer: `[GeneratedRegex]` causes the .NET regular-expression source generator to create a specialized regex implementation at compile time. This avoids repeatedly parsing the pattern and constructing a new `Regex` object at runtime, can improve startup and matching performance, and can provide compile-time diagnostics for invalid patterns.

    It is especially useful for known, constant patterns that are used repeatedly:

    ```
    [GeneratedRegex(@"^\d+$")]
    private static partial Regex DigitsRegex();
    ```

13. When would you choose a `Dictionary<TKey,TValue>` instead of a `List<T>`?

    Answer: Choose `Dictionary<TKey, TValue>` when each value has a unique key and the program frequently needs to retrieve values by that key. Dictionary lookup by key normally approaches constant time:

    ```
    Dictionary<string, Customer> customersById = [];
    Customer customer = customersById["ALFKI"];
    ```

    Choose `List<T>` when the values primarily form an ordered sequence, are accessed by numeric position, can contain duplicates, or are normally processed by enumeration.

14. How do the processing orders of a queue, stack, and priority queue differ?

    Answer: A queue processes elements in **first-in, first-out**, or FIFO, order. The item that has waited longest is removed first.

    A stack processes elements in **last-in, first-out**, or LIFO, order. The most recently added item is removed first.

A priority queue processes elements according to their priorities rather than solely according to insertion order. By default, .NET's `PriorityQueue<TElement, TPriority>` removes the item with the lowest priority value first.

15. What do the `with(...)` and `..` elements do in a collection expression?

    Answer: A `with(...)` element passes arguments to the collection's constructor or builder method. It must be the first element and can specify values such as an initial capacity or equality comparer:

    ```
    List<string> names =
        [with(capacity: 100), "Alice", "Bob"];

    HashSet<string> codes =
        [with(StringComparer.OrdinalIgnoreCase), "abc", "ABC"];
    ```

    A spread element, written with `..`, inserts all the elements from another enumerable collection into the new collection:

    ```
    int[] first = [1, 2];
    int[] second = [3, 4];

    int[] combined = [0, .. first, .. second, 5];
    // 0, 1, 2, 3, 4, 5
    ```

16. Why should you *not* use the official standard for email addresses to create a regular expression to validate a user's email address?

    Answer: The official email-address standards permit many unusual and complicated forms. A regular expression that attempts to implement the complete standard becomes difficult to understand, test, maintain, and protect against pathological input. A restrictive expression can also reject valid addresses, while a permissive expression can accept addresses that do not exist.

    Use a small, practical check to catch obvious structural errors, then send a confirmation message containing a link or code. Only successful delivery and confirmation provide useful evidence that the user controls the address.

Chapter 9 — Processes, Files, Streams, and Serialization

Exercise 9.3 — Test your knowledge

Suggested answers to these questions:

1. What is the difference between a process, a thread, and a `Task`?

   Answer: A **process** is a running application together with the operating-system resources assigned to it. A **thread** is a path of execution within a process and is scheduled by the operating system. A `Task` is a higher-level .NET abstraction representing an operation that is in progress or will complete later. A task might run code on a thread-pool thread, but it does not necessarily have its own thread and might occupy no thread while waiting for asynchronous I/O.

2. Why should a program check the exit code after running an external command-line tool?

   Answer: An external tool's exit code indicates whether it believes the operation succeeded. By convention, zero usually means success and a nonzero value indicates an error, although the exact meanings are defined by each tool. A program that ignores the exit code might continue as though a failed compilation, conversion, deployment, or other operation had succeeded.

3. Why can redirecting both `stdout` and `stderr` cause a deadlock if the streams are not read correctly?

   Answer: Redirected standard-output and standard-error streams use buffers of limited size. If the parent reads one stream synchronously while the child fills the other stream's buffer, the child can block waiting for space. The parent can then wait for the child, producing a deadlock. Read both streams concurrently, normally using asynchronous operations, or use an API that handles both streams for you.

4. What is the benefit of using `await process.WaitForExitAsync()` instead of `process.WaitForExit()`?

   Answer: `process.WaitForExit()` blocks the current thread until the process exits. Awaiting `process.WaitForExitAsync()` returns control to the caller while the process is running, allowing a user interface to remain responsive or a server thread to perform other work. The asynchronous method also accepts a cancellation token.

5. When would you use `Process.RunAndCaptureTextAsync` instead of manually creating a `ProcessStartInfo` and reading `StandardOutput`?

   Answer: Use `Process.RunAndCaptureTextAsync` for a one-shot operation in which you want to start a command, wait for it, and collect its standard output, standard error, and exit status. It avoids manually configuring redirection and coordinating reads from both streams.

```
ProcessTextOutput result =
    await Process.RunAndCaptureTextAsync(
        "git",
        ["status", "--porcelain"]);
```

```
Console.WriteLine(result.StandardOutput);
Console.WriteLine(result.StandardError);
Console.WriteLine(result.ExitStatus.ExitCode);
```

Create and manage a `Process` manually when you need interactive input, streaming output as it arrives, unusual lifetime management, or finer control over events and process settings.

6. Why is it safer to pass process arguments separately rather than building one command string?

   Answer: Passing arguments separately allows .NET to apply the correct quoting and escaping rules to each argument. This prevents spaces, quotation marks, and other characters in one value from accidentally turning it into several arguments or command options.

   ```
   ProcessStartInfo startInfo = new("tool");
   startInfo.ArgumentList.Add("--output");
   startInfo.ArgumentList.Add(filePath);
   ```

   This is safer and more reliable than manually constructing one command string, although untrusted arguments must still be validated.

7. Why should C# code use `Path.Combine` or related `Path` methods instead of hardcoding directory separator characters?

   Answer: Directory separators and path rules differ between operating systems. `Path.Combine`, `Path.Join`, and related methods use platform-appropriate separators and handle path components more reliably than manually inserting `\` or `/`. Hardcoding a Windows backslash can produce an invalid or unintended path on Unix-based systems.

8. What is the difference between using the static `File` class and creating a `FileInfo` instance?

   Answer: The static `File` class is convenient for performing a single operation when you already have a path:

   ```
   string text = File.ReadAllText(path);
   ```

   A `FileInfo` instance represents a particular file and provides instance methods and properties for repeatedly examining or manipulating it:

   ```
   FileInfo file = new(path);

   Console.WriteLine(file.Length);
   file.CopyTo(backupPath);
   ```

   `File` methods require a path for each operation, whereas a `FileInfo` object retains its path and is often more convenient when several operations concern the same file.

9. What does `FileShare.Read` mean when opening a file?

   Answer: `FileShare.Read` means that while your file handle remains open, other processes or streams are permitted to open the same file for reading. They are not permitted to open it for writing or deletion unless you also specify the corresponding sharing flags. `FileShare` controls what access you grant to other users of the file, not what your own stream can do.

10. What is a stream, and why are streams useful even when the data does not come from a file?

    Answer: A stream is an abstraction representing a sequence of bytes that can be read, written, or both. Streams allow the same programming model to work with files, memory, network connections, pipes, compressed data, encrypted data, and process input or output. Code can therefore process data without necessarily knowing where the bytes originated or where they will be stored.

11. What is the difference between the `ReadByte` method and the `Read` method of a stream?

    Answer: `ReadByte` reads one byte and returns it as an int from `0` through `255`, or returns `-1` at the end of the stream.

    ```
    int value = stream.ReadByte();
    ```

    `Read` attempts to read several bytes into a buffer and returns the number actually read. It returns `0` at the end of the stream and can return fewer bytes than requested.

    ```
    int bytesRead = stream.Read(buffer);
    ```

    `Read` is normally preferable for processing blocks of data efficiently, while `ReadByte` is convenient when examining individual bytes.

12. What is the difference between a storage stream, a function stream, and a stream helper?

    Answer: A **storage stream**, also called a **base stream**, reads bytes from or writes bytes to a source or destination such as a file, memory block, pipe, or network connection. Examples include `FileStream` and `MemoryStream`.

    A **function stream**, also called a **pass-through** or **decorator stream**, wraps another stream and transforms or manages its bytes. Examples include `DeflateStream`, `CryptoStream`, and `BufferedStream`.

    A **stream helper** provides a higher-level way to read or write data through a stream. For example, `StreamReader` and `StreamWriter` work with text, while `BinaryReader` and `BinaryWriter` work with primitive values. Stream helpers do not normally represent the underlying storage themselves.

13. Why should you usually compress data before encrypting it?

    Answer: You should normally compress data before encrypting it because compression finds repeated patterns, but strong encryption makes output appear random and therefore difficult to compress. Compressing first usually produces a smaller result and means that the encryption algorithm has fewer bytes to process.

For network protocols, compression must be designed carefully because combining attacker-controlled and secret data in the same compressed-and-encrypted response can create compression side-channel vulnerabilities.

14. When would you use the `StringReader`, `TextReader`, and `StreamReader` classes?

    Answer: Use `StringReader` when the text is already held in a `string` and an API expects a text reader. Use `StreamReader` when text must be decoded from a byte stream, such as a file or network stream, using an appropriate character encoding.

    `TextReader` is the abstract base class of both. Use `TextReader` as a parameter or variable type when your code should accept text from either a string, stream, or another character source without depending on the specific implementation.

15. What does the `DeflateStream` type do?

    Answer: `DeflateStream` wraps another stream and compresses or decompresses bytes using the Deflate algorithm. In compression mode, bytes written to the `DeflateStream` are compressed into the underlying stream. In decompression mode, bytes read from it are decompressed from the underlying stream.

16. What is the difference between compression and serialization?

    Answer: **Compression** reduces the number of bytes needed to represent existing data. Decompression restores the original bytes.

    Serialization converts an object or object graph into a representation that can be stored or transmitted, such as JSON or XML. Deserialization reconstructs data or objects from that representation.

    Serialization changes the form of the data, while compression attempts to reduce its size. Serialized JSON can subsequently be compressed, so the two operations are complementary rather than alternatives.

17. Why can ASCII fail to represent text that UTF-8 can represent?

    Answer: ASCII defines only a small set of characters, including English letters, digits, punctuation, and control characters. It cannot represent most of the world's writing systems or many modern symbols and emoji. UTF-8 can encode every Unicode scalar value while using the same single-byte values as ASCII for the basic ASCII character range.

18. What is an object graph, and why might you serialize one to JSON?

    Answer: An **object graph** is a collection of objects connected through references, such as a `Customer` containing an `Address` and a collection of `Order` objects. Serializing the graph to JSON converts its relevant data into text that can be stored in a file, sent through a web API, placed on a message queue, or consumed by another application.

    Special configuration might be needed when the graph contains cycles, repeated references, or polymorphic objects.

19. Why does `System.Text.Json` not always deserialize properties with private setters unless you configure the type appropriately?

    Answer: `System.Text.Json` builds a serialization contract that determines which members it can read and write. By default, it works primarily with public properties and public accessors, so it does not automatically bypass a private setter merely because the property exists.

    A property with a private setter can be included in reflection-based serialization by applying `[JsonInclude]`, or its value can be supplied through an appropriate constructor, potentially marked with `[JsonConstructor]`. Source-generated serialization has tighter restrictions and does not support private accessors through `[JsonInclude]`.

20. Why is JSON usually better than XML for web APIs, and when might XML still be a better choice?

    Answer: JSON is usually preferred for web APIs because typical object-and-collection payloads are relatively compact, it maps naturally to JavaScript data structures, and it is widely supported by clients and web frameworks. ASP.NET Core uses JSON as its default API format.

    XML can be a better choice when an existing system or industry standard requires it, or when the data benefits from XML-specific features such as namespaces, attributes, mixed text and elements, formal schemas, XPath, XSLT, or established SOAP-based integrations. ASP.NET Core can support XML by adding its XML formatters.

Chapter 10 — Working with Data Using Entity Framework Core

Exercise 10.3 — Test your knowledge

Suggested answers to these questions:

1. What is an ORM?

   Answer: An **object-relational mapper (ORM)**, maps data in a relational database to objects in an application. It allows developers to query and update data using .NET types and expressions instead of writing most SQL and low-level data-access code manually. EF Core is an ORM for .NET.

2. What is the role of a `DbContext` class in EF Core?

   Answer: A `DbContext` represents a short-lived session with a database. It holds the EF Core model, provides access to entity sets, coordinates queries, tracks changes to entity objects, and writes those changes to the database when `SaveChanges` or `SaveChangesAsync` is called. A context normally represents one unit of work and should be disposed afterward.

3. What does a `DbSet<Product>` property usually represent?

   Answer: A `DbSet<Product>` represents the set of `Product` entities that EF Core can query and save. With a relational database, it usually corresponds to a table or another queryable database object containing product data, although the relationship between a `DbSet<T>` and a table is not always strictly one-to-one.

4. What is the difference between database-first and code-first EF Core development?

   Answer: In **database-first** development, the database schema is the source of truth. EF Core reverse-engineers the existing database to generate entity classes, a `DbContext`, and model configuration.

   In **code-first** development, the C# entity classes and model configuration are the source of truth. EF Core migrations are normally used to create the database and apply subsequent schema changes.

5. Why does this book use SQLite for most examples?

   Answer: This book uses SQLite for most examples because it is lightweight, cross-platform, easy to set up, and does not require a separate database server. A SQLite database can be stored in a single file, making the examples easy to create, copy, reset, and run on Windows, macOS, and Linux.

   SQLite is suitable for learning common EF Core concepts, as well as being used in production environments, although provider differences mean that an application intended for SQL Server, PostgreSQL, or another production database should also be tested against that database system.

6. Why might you separate entity model classes from the database context class?

   Answer: Separating entity classes from the `DbContext` class keeps the data objects independent from database configuration and connection management. This can make the entity model easier to reuse, test, navigate, and maintain. It also allows scaffolded or generated context code to be replaced without unnecessarily changing handwritten entity behavior.

   In a larger application, the entity classes and context might also be placed in separate projects or architectural layers, although unnecessary separation can make a small application more complicated.

7. What does the `dotnet ef dbcontext scaffold` command do?

   Answer: The `dotnet ef dbcontext scaffold` command reads an existing database schema and generates a derived `DbContext`, entity classes, relationships, and model configuration. This process is called scaffolding or reverse engineering. The command needs a connection string and an EF Core database provider.

8. Why are scaffolded entity classes often declared as `partial`?

   Answer: Scaffolded entity classes are often declared `partial` so that handwritten members can be added in separate source files. When the database is scaffolded again, EF Core can replace the generated file without overwriting the separate handwritten part of the class. Scaffolded `DbContext` classes are also normally partial for the same reason.

9. What are the three main ways to configure an EF Core model?

   Answer: The three main ways to configure an EF Core model are:

   - **Conventions**, which recognize common patterns automatically.
   - **Mapping attributes**, also called **data annotations**, which are placed on classes and properties.
   - The **Fluent API**, which uses `ModelBuilder` calls, normally in `OnModelCreating`.

   Data annotations override conventions, and Fluent API configuration can override both conventions and data annotations.

10. Why might `[StringLength]` be useful on an entity property?

    Answer: `[StringLength]` specifies the maximum length of a string and can optionally specify a minimum length:

    ```
    [StringLength(40, MinimumLength = 2)]
    public string Name { get; set; } = null!;
    ```

    The maximum length can influence the database column type, such as producing `nvarchar(40)` on SQL Server. Validation frameworks such as ASP.NET Core can also use the attribute to reject invalid input. The minimum length affects validation but does not affect the database schema. EF Core itself does not necessarily validate the length before sending a value to the database.

11. Why might the Fluent API be required even when data annotations are used?

    Answer: Data annotations cover many common cases, but they cannot express every EF Core mapping. The Fluent API might be required for advanced relationships, delete behavior, shadow properties, value conversions, table splitting, owned types, provider-specific settings, or other detailed configuration.

    The Fluent API is also useful when you do not want persistence-related attributes in your entity classes. When both approaches configure the same feature, the Fluent API takes precedence over the data annotation.

12. What does deferred execution mean in a LINQ to EF Core query?

    Answer: Deferred execution means that constructing a LINQ query does not immediately send it to the database. Each LINQ call builds an expression representing the query. EF Core translates and executes it only when the results are consumed:

    ```
    IQueryable<Product> query = context.Products
        .Where(product => product.UnitPrice > 20);

    // The database query has not normally run yet.

    List<Product> products = await query.ToListAsync();

    // The query is executed here.
    ```

Enumerating the same query again can execute another database command and might return different data if the database has changed.

13. Which common LINQ operations usually execute a query immediately?

    Answer: Operations that materialize or consume the results normally execute the query immediately. Common examples include:

    ```
    ToList        ToArray
    First         FirstOrDefault
    Single        SingleOrDefault
    Count         LongCount
    Any           All
    Min           Max
    Sum           Average
    ```

    Their asynchronous equivalents, such as `ToListAsync`, `SingleAsync`, and `CountAsync`, also execute the query. Iterating with `foreach` or `await foreach` causes execution as well. Operations such as `Where`, `Select`, `OrderBy`, and `Take` normally continue building the query rather than executing it, including chaining multiple `Where` calls.

14. What does the `Include` method do?

    Answer: `Include` requests eager loading of related data. For example, the following query loads products and populates each product's related `Category` navigation property:

    ```
    List<Product> products = await context.Products
        .Include(product => product.Category)
        .ToListAsync();
    ```

    `ThenInclude` can load additional levels of related data. Including several collection navigations can produce large joins or a cartesian explosion, so related data should be included only when it is needed.

15. Why should `FromSqlRaw` be used more carefully than `FromSql`?

    Answer: `FromSql` accepts an interpolated SQL string and automatically converts interpolated values into database parameters:

    ```
    string city = "London";

    List<Customer> customers = await context.Customers
        .FromSql($"SELECT * FROM Customers WHERE City = {city}")
        .ToListAsync();
    ```

`FromSqlRaw` accepts raw SQL text. If untrusted values are concatenated or interpolated into that text, the application can be vulnerable to SQL injection:

```
// Unsafe when city came from an untrusted source.
.FromSqlRaw(
    $"SELECT * FROM Customers WHERE City = '{city}'")
```

Use `FromSqlRaw` only when raw SQL construction is genuinely necessary, and parameterize values or carefully validate structural elements that cannot be parameterized, such as dynamically selected column names.

16. What is the EF Core convention for primary keys?

    Answer: By convention, EF Core treats a property named `Id` or `<type-name>Id` as the entity's primary key. For example, both `Id` and `ProductId` would be recognized as the key of a `Product` entity.

    A differently named property must normally be configured with `[Key]` or the Fluent API. Numeric and `Guid` primary keys are also configured for generated values by convention when appropriate.

17. When might you use an annotation attribute in an entity class?

    Answer: Use an annotation attribute when a class or property does not follow EF Core conventions or when simple mapping or validation information is clearest beside the member it affects. Examples include:

```
[Table("People")]
public class Person
{
    [Key]
    [Column("PersonIdentifier")]
    public int Id { get; set; }

    [Required]
    [StringLength(50)]
    public string Name { get; set; } = null!;
}
```

    Attributes are convenient for straightforward configuration. The Fluent API is usually preferable when the configuration is complicated, provider-specific, or should be kept outside the entity classes.

18. How should you define an EF Core entity class to match the following table?

```
CREATE TABLE Employees(
  EmpId INT IDENTITY PRIMARY KEY,
  FirstName NVARCHAR(40) NOT NULL,
```

```
  Salary MONEY
)
```

Answer: The corresponding entity class could be defined as follows:

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

[Table("Employees")]
public partial class Employee
{
    [Key]
    public int EmpId { get; set; }

    [StringLength(40)]
    public string FirstName { get; set; } = null!;

    [Column(TypeName = "money")]
    public decimal? Salary { get; set; }
}
```

`EmpId` needs `[Key]` because it does not match the `Id` or `EmployeeId` naming convention. `FirstName` is non-nullable and limited to 40 characters. `Salary` is `decimal?` because the SQL column permits `NULL`, and its column type is explicitly mapped as `money`.

Chapter 11 — Querying and Manipulating Data Using LINQ

Exercise 11.3 — Test your knowledge

Suggested answers to these questions:

1. What are the required components of LINQ, and which components are optional?

   Answer: A complete LINQ operation has three main components:

   - A **data source**, such as an array, list, XML document, or EF Core `DbSet<T>`.
   - A **query definition** that describes what data to retrieve or transform.
   - **Query execution**, normally caused by enumeration or an operation that produces an immediate result.

   Filtering, ordering, grouping, joining, and transforming elements into a different shape are optional. A query can use only the operations needed for its purpose.

2. What is the difference between method syntax and query comprehension syntax?

   Answer: Method syntax expresses a query as calls to LINQ extension methods, usually combined with lambda expressions:

   ```
   IEnumerable<Product> products = source
       .Where(product => product.Price > 10)
       .OrderBy(product => product.Name);
   ```

   Query comprehension syntax, usually shortened to query syntax, uses clauses such as `from`, `where`, `orderby`, and `select`:

   ```
   IEnumerable<Product> products =
       from product in source
       where product.Price > 10
       orderby product.Name
       select product;
   ```

   The compiler translates query syntax into method calls, so equivalent queries normally have the same meaning and performance. Some LINQ operations have no query-syntax clause and must be written using method syntax.

3. What does deferred execution mean?

   Answer: Deferred execution means that defining a query does not immediately process its data source. The query normally runs later when its results are enumerated, such as by a `foreach` statement.

   Because the source is read at execution time, changing the source before enumeration can change the results. Enumerating the same query more than once can also repeat expensive work or execute another database command.

4. Name three operations that cause a LINQ query to execute immediately.

   Answer: Code statements that cause immediate execution include `ToList()`, `ToArray()`, and `Count()`. Other examples include `First`, `Single`, `Any`, `Sum`, `Average`, `Min`, and `Max`. Enumerating the query with `foreach` also causes it to execute. "Operations" could include actions like opening the **Result view** on a query variable in Visual Studio debug tools or in other code editors.

5. What is the difference between `Where` and `Select`?

   Answer: `Where` filters a sequence. It tests each element and retains only those for which its predicate returns `true`:

   ```
   IEnumerable<Product> expensive =
       products.Where(product => product.Price > 100);
   ```

`Select` projects or transforms every element into a result:

```
IEnumerable<string> names =
    products.Select(product => product.Name);
```

`Where` decides which elements remain, while `Select` decides what each resulting element looks like.

6. What problem does `SelectMany` solve?

   Answer: `SelectMany` solves the problem of working with nested sequences. It projects each source element to a sequence and then flattens all those sequences into one sequence:

   ```
   IEnumerable<OrderLine> lines = orders
       .SelectMany(order => order.OrderLines);
   ```

   Using `Select` here would produce a sequence of sequences, such as `IEnumerable<IEnumerable<OrderLine>>`. `SelectMany` produces one `IEnumerable<OrderLine>`.

7. Why should `ThenBy` normally be used instead of calling `OrderBy` a second time?

   Answer: `ThenBy` adds a secondary ordering that is applied when two elements have the same primary sort key:

   ```
   var people = source
       .OrderBy(person => person.LastName)
       .ThenBy(person => person.FirstName);
   ```

   Calling `OrderBy` a second time starts a new primary ordering and normally replaces the significance of the first ordering:

   ```
   var people = source
       .OrderBy(person => person.LastName)
       .OrderBy(person => person.FirstName);
   ```

   The second example is primarily ordered by `FirstName`, not by `LastName` followed by `FirstName`.

8. What is the difference between `IEnumerable<T>` and `IQueryable<T>`?

   Answer: `IEnumerable<T>` represents a sequence that is enumerated using .NET code. LINQ operators normally accept delegates and run as LINQ to Objects in the current process.

   `IQueryable<T>` represents a query handled by an `IQueryProvider`. Its operators normally build an expression tree that a provider, such as EF Core, can inspect and translate into another query language such as SQL.

   `IQueryable<T>` derives from `IEnumerable<T>`, but its main purpose is to let a provider interpret the query rather than immediately running the lambda expressions as ordinary .NET code.

9. What does calling `AsEnumerable` in the middle of an EF Core query change?

   Answer: Calling `AsEnumerable` changes the provider boundary. Operators before `AsEnumerable` remain part of the `IQueryable<T>` query and can be translated by EF Core. Operators after it bind to `Enumerable` methods and run as LINQ to Objects in the application:

   ```
   var query = context.Products
       .Where(product => product.Price > 10) // Translated to SQL.
       .AsEnumerable()
       .Where(product => CustomTest(product)); // Runs in .NET.
   ```

   `AsEnumerable` does not itself create a list or immediately read all the results. Execution is still deferred until enumeration, but subsequent processing occurs on the client. This should be used carefully because it can transfer more rows from the database than necessary.

10. What is the difference between `Union` and `Concat`?

    Answer: `Concat` places one sequence after another and preserves all elements, including duplicates:

    ```
    [1, 2, 2].Concat([2, 3])
    // 1, 2, 2, 2, 3
    ```

    `Union` performs a set union and returns distinct elements according to the applicable equality comparer:

    ```
    [1, 2, 2].Union([2, 3])
    // 1, 2, 3
    ```

    The result of `Union` therefore depends on the type's equality semantics or on a custom comparer supplied to the operation.

11. How does `OfType<T>` differ from `Cast<T>`?

    Answer: `OfType<T>` returns only elements that are compatible with `T`, silently skipping incompatible elements:

    ```
    IEnumerable<string> strings = objects.OfType<string>();
    ```

    `Cast<T>` attempts to cast every element to `T`. If an element cannot be cast, enumeration throws an `InvalidCastException`:

    ```
    IEnumerable<string> strings = objects.Cast<string>();
    ```

    Use `OfType<T>` to filter a mixed sequence by type. Use `Cast<T>` when every element is expected to have the requested type and an incompatible element represents an error.

12. What is the difference between `Join` and `GroupJoin`?

    Answer: `Join` performs an inner join and produces one flat result for every matching pair of elements:

    ```
    var results = customers.Join(
        orders,
        customer => customer.Id,
        order => order.CustomerId,
        (customer, order) => new { customer, order });
    ```

    A customer with three matching orders produces three results. An element with no match does not appear.

    `GroupJoin` produces one result for each element in the outer sequence, together with a sequence containing all its matching inner elements:

    ```
    var results = customers.GroupJoin(
        orders,
        customer => customer.Id,
        order => order.CustomerId,
        (customer, customerOrders) =>
            new { customer, customerOrders });
    ```

    It is useful for hierarchical results and can form the basis of a left outer join.

13. When would you use `ToLookup` instead of `GroupBy`?

    Answer: Use `ToLookup` when you want to process the source immediately and create an in-memory, dictionary-like grouping that can be repeatedly accessed by key:

    ```
    ILookup<int, Product> productsByCategory =
        products.ToLookup(product => product.CategoryId);

    IEnumerable<Product> categoryProducts =
        productsByCategory[5];
    ```

    Unlike a dictionary, a lookup can associate multiple values with the same key, and requesting a missing key returns an empty sequence.

    `GroupBy` is normally deferred and is suitable when you want to continue composing or enumerate grouped query results. `ToLookup` is suitable when you want the groups materialized now and expect repeated key-based access.

14. Why might either argument passed to a `FullJoin` result selector be `null`?

    Answer: A full join includes matched pairs and unmatched elements from both sequences. When an element from the left sequence has no match, the right argument passed to the result selector is `default`. When an element from the right sequence has no match, the left argument is `default`.

    For reference types, `default` is `null`, so either argument might be null:

    ```
    var results = products.FullJoin(
        categories,
        product => product.CategoryId,
        category => category.Id,
        (product, category) => new
        {
            ProductName = product?.Name ?? "(no product)",
            CategoryName = category?.Name ?? "(no category)"
        });
    ```

    For a non-nullable value type, the missing value is its normal default value, such as `0`, rather than `null`.

15. How can `ToQueryString` or LINQPad help you evaluate an EF Core query?

    Answer: `ToQueryString` shows the SQL representation generated for an EF Core `IQueryable` without executing the query:

    ```
    IQueryable<Product> query = context.Products
        .Where(product => product.Price > 100);

    Console.WriteLine(query.ToQueryString());
    ```

    You can inspect the SQL to check whether filtering and projection occur in the database, whether unnecessary columns or joins are included, and whether the query is likely to produce excessive rows. The returned text is intended for debugging and is not guaranteed to be directly executable SQL.

    LINQPad provides an interactive environment in which you can run LINQ queries, inspect their results, and examine the SQL produced by a database provider. This makes it useful for experimenting with alternative query forms before putting them into an application. Ensure LINQPad is using the same provider, such as EF Core, as the application because different providers can translate the same LINQ query differently.

Chapter 12 – Building Web Apps Using Blazor

Exercise 12.3 – Test your knowledge

Suggested answers to these questions:

1. Why can a Blazor Web App created with Interactive Auto contain both a server project and a client project?

   Answer: A Blazor Web App created with Interactive Auto can contain a server project and a client project because it must support both server-side and WebAssembly interactivity. The server project hosts the application, performs server rendering, and provides Interactive Server execution. The `.Client` project configures the Blazor WebAssembly host and contains components and code that must be downloaded and run in the browser.

2. What are the four Blazor render modes, and where does interactive component code execute in each one?

   Answer: The four Blazor render modes are:

   - **Static Server:** The component executes on the server to produce HTML, but it is not interactive after the response reaches the browser.
   - **Interactive Server:** Component code executes on the server. Browser events and UI updates travel over a real-time Blazor circuit using SignalR.
   - **Interactive WebAssembly:** Component code executes in the browser using the WebAssembly-based .NET runtime.
   - **Interactive Auto:** Component code initially executes interactively on the server and normally executes in the browser on later visits after the WebAssembly runtime and application bundle have been downloaded and cached.

3. How does Interactive Auto normally behave on a first visit compared with a later visit?

   Answer: On the first visit, Interactive Auto normally uses Interactive Server so the component becomes interactive without waiting for the WebAssembly runtime and application bundle to download. Those resources are downloaded and cached in the background. On a later visit, the component normally runs using WebAssembly in the browser. It does not switch an existing component from server execution to WebAssembly while that component remains on the page.

4. How is streaming rendering different from an interactive render mode?

   Answer: Streaming rendering is a server-rendering technique, not an interactive render mode. It sends an initial page containing placeholder content before long-running asynchronous work has finished. When the work completes, additional HTML is sent through the same response and patched into the page.

   Streaming rendering can improve perceived loading time, but it does not by itself keep a component running or enable Blazor event handlers such as `@onclick`. Interactive render modes establish an ongoing execution environment in which component state and event handling continue after the initial HTML has been rendered.

5. What roles do `App.razor` and `Routes.razor` perform?

   Answer: `App.razor` is the root component and normally defines the application's HTML document shell. It typically includes the `<html>`, `<head>`, and `<body>` elements, stylesheets, static assets, the `HeadOutlet`, the `Routes` component, and the Blazor startup script.

   `Routes.razor` contains the `Router` component. It locates the routable component whose route template matches the requested URL and renders it, normally inside the application's default layout.

6. How does the `@page` directive turn an ordinary component into a routable page component?

   Answer: The `@page` directive supplies a route template for a component:

   ```
   @page "/customers"
   ```

   When the component is compiled, the generated component class receives a `RouteAttribute` containing that template. At runtime, the router discovers components with route attributes and renders the component whose template matches the requested URL. Without `@page`, the component can still be nested and reused, but it cannot be reached directly through routing.

7. How is a route parameter assigned to a component property?

   Answer: Place a named placeholder in the route template and declare a component property with the same name and the `[Parameter]` attribute:

   ```
   @page "/customers/{id}"

   @code {
       [Parameter]
       public string? Id { get; set; }
   }
   ```

   When the router matches a URL such as `/customers/ALFKI`, it assigns `ALFKI` to `Id`. Route parameter names are matched without regard to case, and route constraints can request conversion to types such as `int`, `Guid`, or `DateTime`.

8. What advantage does `NavLink` provide over an ordinary HTML anchor, and what does `NavLinkMatch.All` change?

   Answer: `NavLink` renders an HTML anchor but also compares its destination with the current URL. When they match, it automatically applies an `active` CSS class, allowing navigation menus to visually identify the current page.

   The default `NavLinkMatch.Prefix` considers the link active when its URL is a prefix of the current URL. `NavLinkMatch.All` requires the whole URL path to match, so a home link for `/` does not remain active on every page. In current ASP.NET Core versions, query strings and fragments are normally ignored when `All` performs its comparison.

9. How does a layout decide where to render the body of a page component?

   Answer: A layout normally inherits from `LayoutComponentBase`, which provides a Body property containing the page component's rendered content. The layout places `@Body` wherever that content should appear:

   ```
   @inherits LayoutComponentBase

   <header>
       <NavMenu />
   </header>

   <main>
       @Body
   </main>
   ```

   Everything surrounding `@Body` is shared layout content, while the matched page component is rendered at the `@Body` location.

10. How does `ASPNETCORE_ENVIRONMENT` affect an ASP.NET Core application during startup?

    Answer: ASP.NET Core reads `ASPNETCORE_ENVIRONMENT` when the application starts and exposes the resulting environment name through `IWebHostEnvironment`. Common values are `Development`, `Staging`, and `Production`.

    The environment affects which environment-specific configuration is loaded, such as `appsettings.Development.json`, and allows startup code to conditionally register services or configure middleware. For example, an application might enable detailed exception information in development but use production error handling and security policies in production. If no environment is specified for a deployed application, it normally defaults to `Production`. Changing the value requires restarting the application.

11. When should a component use `OnInitializedAsync` rather than `OnParametersSetAsync`?

    Answer: Use `OnInitializedAsync` for asynchronous initialization that should occur when the component instance is first initialized and does not need to be repeated whenever its parameters change. Examples include loading fixed reference data or initializing a service used for the component's lifetime.

    Use `OnParametersSetAsync` when the operation depends on component or route parameters and must be repeated when those parameters change. Blazor can retain the same component instance while navigating between URLs handled by that component, so `OnInitializedAsync` is not necessarily called again, but `OnParametersSetAsync` is.

12. What does a `.razor.css` file do, and what does CSS isolation not guarantee?

    Answer: A file named after a component with the `.razor.css` suffix contains CSS isolated for that component:

    ```
    CustomerDetail.razor
    CustomerDetail.razor.css
    ```

    During the build, Blazor rewrites the selectors and adds generated scope attributes to the component's rendered HTML. This reduces accidental style collisions without requiring manually invented class names.

    CSS isolation is not a security or complete encapsulation boundary. Global CSS can still affect the component, ordinary CSS cascade and specificity rules still apply, and styles do not automatically reach markup rendered inside child components unless an approach such as `::deep` is used.

13. Why is it useful to place data access behind an interface such as `INorthwindService`?

    Answer: Placing data access behind an interface such as `INorthwindService` separates the component's user-interface code from the mechanism used to retrieve and update data. The component depends on operations such as `GetCustomersAsync` rather than depending directly on EF Core, HTTP, or a particular database.

    Different implementations can therefore be supplied through dependency injection. A server component might use EF Core directly, while a WebAssembly component might call a web API. Tests can provide a fake implementation without accessing a real database. This also keeps database concerns out of Razor components and makes implementations easier to change independently.

14. Why should an interactive server component avoid sharing one `DbContext` across overlapping operations?

    Answer: A `DbContext` is not thread-safe and must not be used for concurrent database operations. An Interactive Server circuit can live much longer than one HTTP request, and different component events or asynchronous operations can overlap. Sharing one context across those operations can cause an `InvalidOperationException`, corrupted tracking state, undefined behavior, or data corruption.

    A common approach is to inject `IDbContextFactory<TContext>` and create one short-lived context for each operation:

    ```
    await using NorthwindDb context =
        await contextFactory.CreateDbContextAsync();

    List<Customer> customers =
        await context.Customers.ToListAsync();
    ```

Each asynchronous EF Core operation should also be awaited before the same context is used again.

15. How do `EditForm`, `DataAnnotationsValidator`, `ValidationMessage`, and `OnValidSubmit` work together?

    Answer: `EditForm` manages the model being edited and its validation state, usually through an `EditContext`:

    ```
    <EditForm Model="customer"
              OnValidSubmit="SaveCustomerAsync">
        <DataAnnotationsValidator />

        <InputText @bind-Value="customer.CompanyName" />
        <ValidationMessage
            For="() => customer.CompanyName" />

        <button type="submit">Save</button>
    </EditForm>
    ```

    `DataAnnotationsValidator` connects validation attributes such as `[Required]` and `[StringLength]` on the model to the form's validation system. `ValidationMessage` displays messages associated with one selected property. When the user submits the form, `EditForm` requests validation. `OnValidSubmit` is called only when validation succeeds, so the handler can save the valid model. If validation fails, the messages are displayed and the valid-submit handler is not called.

Know your webbreviations

What do the following web abbreviations stand for, and what do they do?

1. **URI** (**Uniform Resource Identifier**) is a unique sequence of characters that identifies a resource.
2. **URL** (**Uniform Resource Locator**) is the address of a unique resource.
3. **SSR** (Static **Server Rendering**) generates the page's HTML entirely on the server for each request and sends it to the browser as static, non-interactive markup with no ongoing SignalR connection or client-side re-rendering.
4. **TLD** (**Top-Level Domain**) is one of the domains at the highest level in the hierarchical **DNS** (**Domain Name System**) of the internet, for example, `packt.com`.
5. **API** (**Application Programming Interface**) is a mechanism for a computer system to allow other computer systems to communicate with it. APIs should be well-documented.
6. **SPA** (**Single-Page Application**) is a web app that loads only a single web page and then updates the content dynamically via JavaScript APIs when needed. SPA frameworks include Angular, React, Vue, and Blazor WebAssembly.
7. **CMS** (**Content Management System**) is a software application that allows users to build and manage a website without having to write code.

8. **Wasm** (**WebAssembly**) is a binary instruction format and is designed as a compilation target for programming languages such as C#, for deployment on the web, client, and server.
9. **DI** (**Dependency Injection**) is a design pattern in which a class receives the objects it depends on (its "dependencies") from an external source rather than creating them itself, and Blazor has a built-in DI container that lets you register services and have them automatically supplied to components, typically via constructor or `@inject` parameters.
10. **REST** (**REpresentational State Transfer**) is an architectural style for distributed hypermedia systems. Roy Fielding presented it in 2000 in his famous dissertation.



In case you missed it (**ICYMI**): Acronyms and initialisms are types of abbreviation. Learn more at the following link: `https://www.rd.com/article/acronym-vs-abbreviation-whats-the-difference/`.

Chapter 13 – Building and Consuming Web Services

Exercise 13.3 – Test your knowledge

Suggested answers to these questions:

1. What is the difference between an HTTP API and a RESTful API?

   Answer: An **HTTP API** is any application interface that clients call using HTTP. A **RESTful API** is an HTTP API designed around REST principles, such as identifying resources with URIs, exchanging representations of those resources, using standard HTTP methods according to their intended semantics, keeping requests stateless, and using HTTP status codes consistently. Every RESTful API is an HTTP API, but an HTTP API that merely uses HTTP as a transport is not necessarily RESTful.

2. What purposes do the `GET`, `POST`, `PUT`, `PATCH`, and `DELETE` methods normally serve?

   Answer: The HTTP methods normally serve these purposes:

   - `GET` retrieves a representation of a resource without changing it.
   - `POST` normally creates a resource in a collection or submits data for processing.
   - `PUT` normally completely replaces or updates the resource at a known URI and can sometimes create it.
   - `PATCH` applies a partial update, so the client sends only the changes.
   - `DELETE` removes the resource identified by the URI.

   `GET`, `PUT`, and `DELETE` have idempotent semantics. Repeating the same request should have the same intended effect as sending it once, although the returned status code can differ.

3. Which status code and response header should normally be returned after creating a resource?

   Answer: After successfully creating a resource, an API should normally return `201 Created` and a `Location` response header containing the URI of the newly created resource. The response body can also contain a representation of that resource.

4. When is a `204 No Content` response appropriate?

   Answer: A `204 No Content` response is appropriate when the request succeeded but the server has no representation or other content to return in the response body. It is commonly used after a successful `PUT`, `PATCH`, or `DELETE` operation when the client does not need the updated resource returned. A 204 response must not contain a message body.

5. What is the purpose of a route constraint, and why should it not be used for input validation?

   Answer: A route constraint helps routing distinguish between otherwise similar route templates. For example, `{id:int}` can distinguish a numeric identifier route from one that accepts a name.

   A route constraint should not be used to decide whether an input value is valid for the application. When a constraint does not match, routing treats the endpoint as absent and normally returns `404 Not Found`. Invalid input for a matched endpoint should instead produce `400 Bad Request` with an explanation of the problem.

6. From which parts of an HTTP request can Minimal API parameters be bound?

   Answer: Minimal API parameters can be bound from:

   - Route values.
   - Query-string values.
   - HTTP headers.
   - A JSON request body.
   - Form fields and uploaded files.

   A handler can also receive services from dependency injection and values supplied through custom binding mechanisms. Framework types such as `HttpContext`, `HttpRequest`, `HttpResponse`, and `CancellationToken` can also be supplied directly to handlers.

7. How does ASP.NET Core respond when built-in Minimal API validation fails?

   Answer: After built-in Minimal API validation has been enabled with `AddValidation`, ASP.NET Core validates supported parameters before calling the endpoint handler. If validation fails, it normally returns `400 Bad Request` with a problem-details response containing the validation errors. The response formatting can be customized through `IProblemDetailsService`.

8. What advantages does `TypedResults` have over the untyped `Results` helpers?

   Answer: The untyped `Results` helpers return the general `IResult` interface. `TypedResults` helpers return concrete result types such as `Ok<Product>`, `NotFound`, or `Created<Product>`.

   The concrete types improve compile-time checking and make unit tests easier because tests do not have to cast an `IResult` to discover what was returned. They also automatically contribute response type, status code, and content metadata to the generated OpenAPI document.

9. Why should an endpoint that has multiple possible typed outcomes return `Results<T1, ... TN>` instead of `IResult`?

   Answer: When an endpoint has several possible typed outcomes, `Results<T1, ... TN>` preserves all the declared result types in the method signature:

   ```
   static async Task<Results<Ok<Product>, NotFound>>
       GetProductAsync(int id, ProductDb db)
   {
       Product? product = await db.Products.FindAsync(id);

       return product is null
           ? TypedResults.NotFound()
           : TypedResults.Ok(product);
   }
   ```

   This allows every declared result to contribute OpenAPI metadata and lets the compiler reject an attempt to return an undeclared result type. Returning only `IResult` erases that concrete type information, so metadata often has to be added manually with methods such as `Produces`.

10. What information does a `ProblemDetails` response provide to an API client?

    Answer: A `ProblemDetails` response provides a standard, machine-readable description of an HTTP API error. Its standard properties include:

    - `type`: A URI identifying the category of problem.
    - `title`: A brief, human-readable summary.
    - `status`: The HTTP status code.
    - `detail`: An explanation of this particular occurrence.
    - `instance`: A URI identifying this occurrence of the problem.
    - `extensions`: Additional application-specific information.

    A validation problem also normally contains an `errors` collection associating parameters or properties with their validation messages.

11. What is the difference between an OpenAPI document name and an API version?

    Answer: An **OpenAPI document name** is an internal identifier used to register, select, and expose one generated document. For example:

    ```
    builder.Services.AddOpenApi("internal");
    ```

    This document can then be exposed as `/openapi/internal.json`. The default name is `v1`, but that name alone does not make the application's API version 1.

    An **API version** describes a version of the public contract consumed by clients, such as version 1 or version 2 of an endpoint. API versioning must be designed and configured separately. There is also a third distinct concept, the **OpenAPI specification version**, such as OpenAPI 3.0, 3.1, or 3.2, which defines the format of the document itself.

12. How can the same OpenAPI document be exposed in both JSON and YAML formats?

    Answer: Register the OpenAPI document once, then map two routes whose suffixes select the serialization format:

    ```
    builder.Services.AddOpenApi();

    WebApplication app = builder.Build();

    if (app.Environment.IsDevelopment())
    {
        app.MapOpenApi(); // /openapi/{documentName}.json

        app.MapOpenApi(
            "/openapi/{documentName}.yaml");
    }
    ```

    For the default document name, the same document is then available as both `/openapi/v1.json` and `/openapi/v1.yaml`. The `.json`, `.yaml`, or `.yml` suffix tells ASP.NET Core which format to produce.

13. What role does Scalar perform, and what role does `Microsoft.AspNetCore.OpenApi` perform?

    Answer: `Microsoft.AspNetCore.OpenApi` generates the OpenAPI document from the application's endpoint metadata. It supports runtime document generation, multiple documents, customization through transformers, and exposing documents through endpoints such as `/openapi/v1.json`.

    Scalar consumes an OpenAPI document and presents it as an interactive browser-based reference and testing interface. Scalar does not generate the API description itself. It displays and interacts with the document produced by `Microsoft.AspNetCore.OpenApi` or another OpenAPI generator.

14. Why does a Blazor WebAssembly app running on port `5132` need a CORS policy to call an API running on port `5131`?

    Answer: A browser origin consists of the scheme, host, and port. Therefore, an app at `https://localhost:5132` and an API at `https://localhost:5131` have different origins even though both use `localhost`.

    The browser's same-origin policy normally prevents the Blazor WebAssembly code from reading responses from the other origin. The API must use a CORS policy that explicitly permits the Blazor application's origin, along with the required methods and headers. The allowed origin must include the correct scheme and port.

15. Why should `HttpClient` instances normally be reused instead of created and disposed for every request?

    Answer: Each `HttpClient` has or uses an underlying connection pool. Repeatedly constructing and disposing clients can prevent efficient connection reuse and leave TCP connections temporarily occupying ports, eventually causing port or socket exhaustion under load.

    Applications should normally use either a long-lived `HttpClient`, with an appropriate `PooledConnectionLifetime` when DNS changes must be recognized, or clients created through `IHttpClientFactory`, which manages and reuses the underlying handlers. The important rule is not to create and dispose an entirely independent client and connection pool for every HTTP request.

This is the end of Appendix A.